

GAUTENG CITY COLLEGE

Advanced Programming II

Data Validation System (Java Swing)

Project title	Data Validation System (Java Swing)
Student name	Emmanuel Chuma
Student number	0508266509084
College	Gauteng City College
Lecturer	Mr Uwami
Date	21 September 2026

Table of contents

Data Validation System (Java Swing)	1
Table of contents	2
1. Introduction	3
2. System design	3
2.1 Colour scheme and layout	3
2.2 Interface sketch / screenshot	4
3. Fields	4
4. Validation checks	4
5. Testing	5
5.1 Before testing - invalid input	6
5.2 After testing - valid input.....	6
5.3 Completed form.....	7
6. Code.....	7
6.1 Validator.java	7
6.2 MainFrame.java - validation methods	8
6.3 MainFrame.java - save/clear logic	8
7. GitHub repository	9
8. Conclusion	10
9. References	10

1. Introduction

The input validation system is a java swing application that checks the input data of multiple fields, ensuring that they meet certain requirements such as Length, format, data type, etc. Its purpose is to ensure that accurate data is entered into the system by the user in EVERY field.

2. System design

2.1 Colour scheme and layout

The interface uses a deliberate two-colour scheme rather than Swing's default grey. The main form panel is a dark teal green (RGB 51, 129, 104), chosen to feel calmer and more professional than a stark white background while still keeping enough contrast for black label text to stay readable. The header banner uses a contrasting burnt orange, which draws the eye to the form's title ("Student Registration Validation") immediately on opening the app. That same orange is reused as the accent colour on input field borders, each text field uses an Etched Border (Raised style) with a highlight colour of RGB 255, 102, 0 and a grey shadow colour of RGB 102, 102, 102, which gives every field a subtle raised, tactile look instead of Swing's flat default outline, and ties the fields visually back to the header.

Action buttons follow a standard colour-coded convention so their purpose is recognisable without reading the label: green for Save (a positive, confirming action), red for Exit (a destructive/final action), and a dark neutral tone for Clear (a reset action that's neither positive nor destructive).

Typography is kept to a single font family throughout, Gill Sans MT 14, for consistency. Field labels (e.g. "Name", "Student number") are bold to distinguish them as instructions, while the small hint text beneath each field (e.g. "Enter Name", "Choose DOB") uses a lighter weight, since it's supporting text rather than a primary instruction. Fields marked with an asterisk (*) indicate required input.

The layout uses field grouping, related fields are placed side by side in pairs (Name/Surname, Student number/Email) rather than in one long vertical list, which keeps the form compact and reduces how far the user has to scroll or scan to complete it. The Comments/Feedback text area is given its own full-width row beneath the paired fields, since it holds significantly more content than a single line. A decorative animated robot graphic sits in the top-right of the form; it's purely aesthetic, placed away from any input field so it doesn't interfere with the tab order or draw attention away from required fields.

2.2 Interface sketch / screenshot

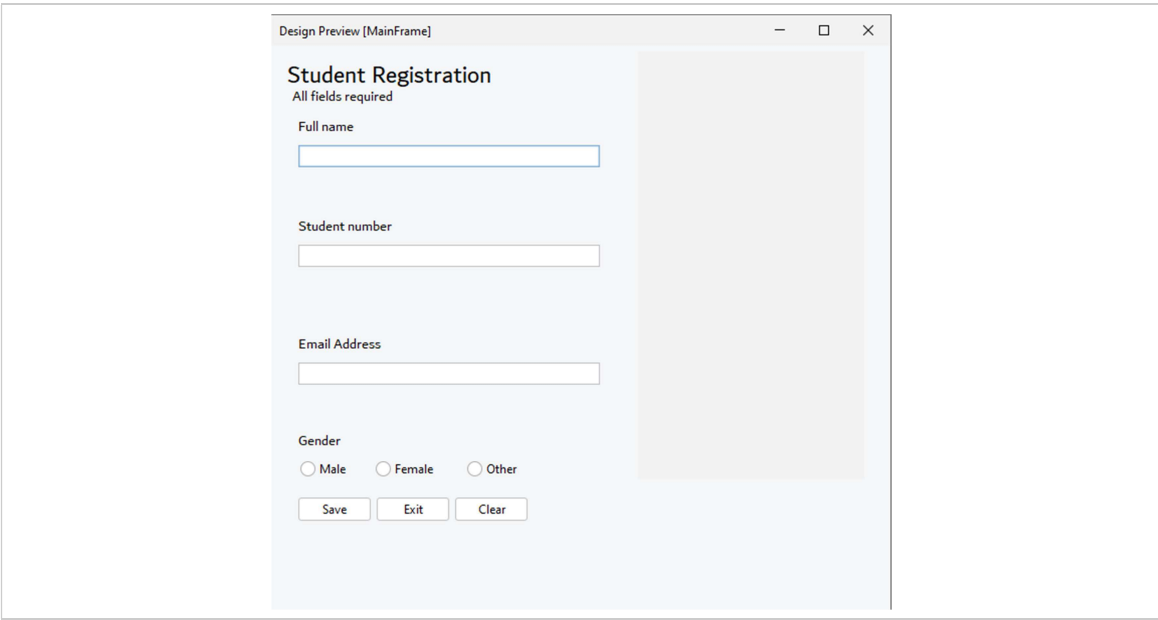


Figure 1: Initial Design Layout

3. Fields

Every field captured by the form and what it is meant to capture.

Field	What it captures
Name	Captures the user's first name (text)
Surname	Captures the user's surname (text)
Student number	Captures the user's college student number (numeric, fixed length)
Email Address	Captures a contactable email address (formatted text)
Cellphone No.	Captures a South African cellphone number (numeric, fixed length)
Date of Birth	Captures date of birth via a date picker, used to calculate age
Gender	Captures gender via three radio buttons - Male, Female, Other
Comments/Feedback	Optional free-text field for additional user comments (not validated)

4. Validation checks

For each field, the checks applied and why they are appropriate. This matches the validateX() methods in MainFrame.java, backed by the static helper methods in Validator.java.

Field	Checks applied	Validator method(s) used
Name	Presence, Type (alphabetic)	isEmpty(), isAlphabetic()
Surname	Presence, Type (alphabetic)	isEmpty(), isAlphabetic()

Student number	Presence, Type (numeric), Length (exactly 8)	isEmpty(), isNumeric(), hasExactLength()
Email Address	Presence, Format (email pattern)	isEmpty(), matchesEmailFormat()
Cellphone No.	Presence, Type (numeric), Length (exactly 10)	isEmpty(), isNumeric(), hasExactLength()
Date of Birth	Presence (date selected), Range (age 16-100)	isInRange() on calculated age
Gender	Presence (one option selected)	checked directly against the radio buttons/ButtonGroup

4.1 Name and Surname — Presence, Type (alphabetic). Both fields are required because a registration record is meaningless without identifying the person. An alphabetic check is appropriate here rather than a length or format check, because a name has no fixed length and no consistent pattern to match against — the only thing that reliably makes an entry invalid is the presence of numbers or symbols, which the alphabetic check catches directly.

Student number — Presence, Type (numeric), Length (exactly 8). A student number is required to identify the applicant within the college's system, so presence is non-negotiable. A type check on its own isn't sufficient here, because "1234" and "20233145" are both numeric but only one is a valid student number — the college issues numbers to a fixed 8-digit format, so an exact-length check is what actually enforces a correct student number rather than just a numeric one.

Email Address — Presence, Format (email pattern). Email is required since it's the system's contact channel for the registered user. A type or length check would be meaningless here: "abcdefgh" is 8 alphabetic characters but not an email, and a numeric check makes no sense for an address containing letters and symbols. Only a format check, matching the general shape of text@text.text, can actually distinguish a valid email from an invalid one.

Cellphone No. — Presence, Type (numeric), Length (exactly 10). Same reasoning as student number: presence and a numeric check alone would still accept something like "12345", so an exact-length check (10 digits, matching a standard South African mobile number) is needed to catch entries that are numeric but incomplete or malformed.

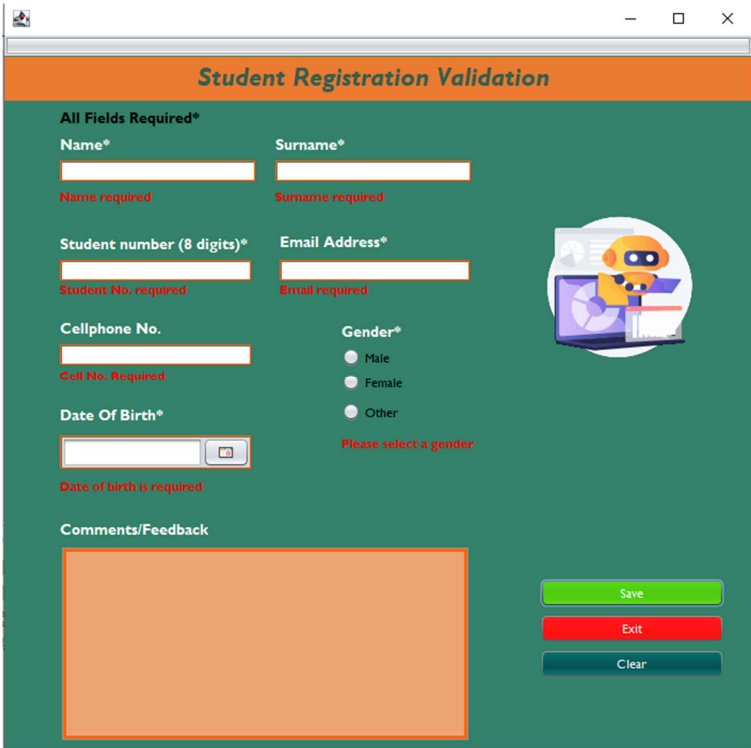
Date of Birth — Presence, Range (age 16-100). A date must be selected before an age can even be calculated, so presence comes first. A range check on the resulting age, rather than a type or format check, is what's actually meaningful here, since the JDateChooser component already guarantees a syntactically valid date; the real risk is a date that produces an implausible age (e.g. an infant or someone over 100 registering), which only a range check on the calculated value can catch.

Gender — Presence (one option selected). Because gender is captured through radio buttons rather than free text, there's no type, length, or format to validate — the only failure state possible is the user leaving all three unselected, so a single presence check against the button group is the only check this field needs.

5. Testing

This is the evidence section and must include screenshots.

5.1 Before testing - invalid input



The screenshot shows a web application window titled "Student Registration Validation". The form has a dark green background with an orange header and footer. The form fields are arranged in two columns. The first column contains "Name*", "Student number (8 digits)*", "Cellphone No.", and "Date Of Birth*". The second column contains "Surname*", "Email Address*", and "Gender*". The "Gender*" field has three radio buttons: "Male", "Female", and "Other". The "Date Of Birth*" field has a date picker. The "Comments/Feedback" field is a large orange text area. The "Save", "Exit", and "Clear" buttons are at the bottom right. The form is in an invalid state, with red error messages below each field: "Name required:", "Surname required:", "Student No. required:", "Email required:", "Cell No. Required:", "Please select a gender", and "Date of birth is required".

Student Registration Validation

All Fields Required*

Name* Surname*
Name required: Surname required:

Student number (8 digits)* Email Address*
Student No. required: Email required:

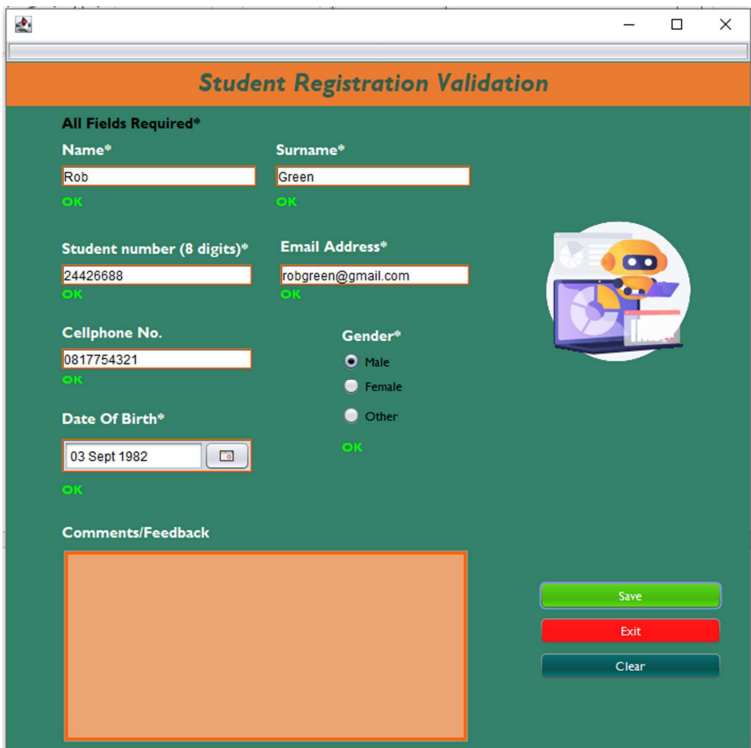
Cellphone No. Gender*
Cell No. Required: ☐ Male
☐ Female
☐ Other
Please select a gender

Date Of Birth*
Date of birth is required

Comments/Feedback

Figure 2: Invalid input testing

5.2 After testing - valid input



The screenshot shows the same web application window as Figure 2, but with valid input. The form fields are now populated with valid data: "Name" is "Rob", "Surname" is "Green", "Student number (8 digits)" is "24426688", "Email Address" is "robgreen@gmail.com", "Cellphone No." is "0817754321", "Date Of Birth" is "03 Sept 1982", and "Gender" is "Male". The "Comments/Feedback" field is empty. The "Save", "Exit", and "Clear" buttons are at the bottom right. The form is in a valid state, with green "OK" messages below each field: "OK", "OK", "OK", "OK", "OK", "OK", and "OK".

Student Registration Validation

All Fields Required*

Name* Surname*
OK OK

Student number (8 digits)* Email Address*
OK OK

Cellphone No. Gender*
OK ☒ Male
☐ Female
☐ Other
OK

Date Of Birth*
OK

Comments/Feedback

Figure 3: Valid Input Testing

5.3 Completed form

The screenshot shows a web application window titled "Student Registration Validation". The form has a green background and an orange header. It contains several input fields, each with a green "OK" message below it, indicating successful validation. The fields are: Name* (Rob), Surname* (Green), Student number (8 digits)* (24426688), Email Address* (robgreen@gmail.com), Cellphone No. (0817754321), Date Of Birth* (03 Sept 1982), and Gender* (Male, selected with a radio button). There is also a "Comments/Feedback" text area. A modal dialog box titled "All details captured successfully" is open, displaying the entered information: Name: Rob, Surname: Green, Student number: 24426688, Email: robgreen@gmail.com, Cellphone: 0817754321, Date of birth: 03/09/1982, and Gender: Male. At the bottom right of the form are three buttons: "Save" (green), "Exit" (red), and "Clear" (blue).

Figure 4: Complete form output

6. Code

6.1 Validator.java

This class holds every reusable validation rule as a static method, so the same check (e.g. `isNumeric`) can be called from multiple fields without duplicating logic. The constructor is purposefully private, since every method is static and no field needs to create an actual `Validator` object, signalling that the class is not a blueprint for objects, rather a toolbox of functions.

```
public class Validator {
    private Validator() {
    }

    public static boolean isEmpty(String value) {
        return value != null && !value.trim().isEmpty();
    }

    public static boolean isAlphabetic(String value) {
        return value != null && value.matches("[a-zA-Z ]+");
    }

    public static boolean isNumeric(String value) {
        return value != null && value.matches("[0-9]+");
    }

    public static boolean hasExactLength(String value, int length) {
        return value != null && value.length() == length;
    }
}
```

```

    public static boolean matchesEmailFormat(String value) {
        return value != null && value.matches("^\\w.+@[\\w-]+\\.\\.[a-zA-Z]{2,}$");
    }

    public static boolean isInRange(int value, int min, int max) {
        return value >= min && value <= max;
    }
}

```

6.2 MainFrame.java - validation methods

Every field on the form has its own validateX() method following the same shape: read the field's value, run it through a sequence of checks from Validator, and update that field's error label. validateStudentNum() below shows this pattern with three checks stacked in a deliberate order, presence first (nothing else is worth checking on an empty field), then type (catching non-numeric input before length is even considered), then exact length (only meaningful once the value is confirmed to be numeric).

```

private boolean validateStudentNum() {
    String value = txtStudentNum.getText().trim();
    if (!Validator.isEmpty(value)) {
        showError(lblStNumError, "Student number is required");
        return false;
    }
    if (!Validator.isNumeric(value)) {
        showError(lblStNumError, "Numbers only");
        return false;
    }
    if (!Validator.hasExactLength(value, 8)) {
        showError(lblStNumError, "Must be exactly 8 digits");
        return false;
    }
    clearError(lblStNumError);
    return true;
}

```

validateDob() follows the same overall shape but works differently under the hood, since it reads from a JDateChooser rather than a text field. "Presence" here means checking whether a date was selected at all (getDate() returning null), and the range check is applied to the applicant's calculated age rather than to raw text.

```

private boolean validateDob() {
    java.util.Date selected = jDateChooser1.getDate();
    if (selected == null) {
        showError(lblDobError, "Date of birth is required");
        return false;
    }
    java.time.LocalDate dob = selected.toInstant()
        .atZone(java.time.ZoneId.systemDefault())
        .toLocalDate();
    int age = java.time.Period.between(dob, java.time.LocalDate.now()).getYears();
    if (!Validator.isInRange(age, 16, 100)) {
        showError(lblDobError, "Age must be between 16 and 100");
        return false;
    }
    clearError(lblDobError);
    return true;
}

```

6.3 MainFrame.java - save/clear logic

saveBtnActionPerformed() calls all seven validate methods joined with the & operator rather than &&. Unlike &&, which stops evaluating as soon as one check fails, & always runs every validate

method regardless of earlier results, so a user who submits a completely empty form sees every field's error at once, rather than one error per click. Only if every method returns true does the form build a summary string and display it in a JOptionPane. clearBtnActionPerformed() resets every field, the date picker, the radio group, and the comments box back to blank, and resets every label to a neutral state (not red or green, since nothing has been validated after a clear).

```
private void saveBtnActionPerformed(java.awt.event.ActionEvent evt) {
    if (validateName() & validateSurname() & validateEmail() & validateStudentNum()
        & validateCell() & validateDob() & validateGender()) {
        String dobText = new
java.text.SimpleDateFormat("dd/MM/yyyy").format(jDateChooser1.getDate());
        String gender;
        if (radioMale.isSelected()) {
            gender = "Male";
        } else if (radioFemale.isSelected()) {
            gender = "Female";
        } else {
            gender = "Other";
        }
        String summary = "Name: " + txtName.getText().trim()
            + "\nSurname: " + txtSurname.getText().trim()
            + "\nStudent number: " + txtStudentNum.getText().trim()
            + "\nEmail: " + txtEmail.getText().trim()
            + "\nCellphone: " + txtCellNo.getText().trim()
            + "\nDate of birth: " + dobText
            + "\nGender: " + gender;
        javax.swing.JOptionPane.showMessageDialog(this, summary,
            "All details captured successfully",
            javax.swing.JOptionPane.INFORMATION_MESSAGE);
    }
}

private void clearBtnActionPerformed(java.awt.event.ActionEvent evt) {
    txtName.setText("");
    txtSurname.setText("");
    txtEmail.setText("");
    txtStudentNum.setText("");
    txtCellNo.setText("");
    jDateChooser1.setDate(null);
    genderBtn.clearSelection();
    jTextArea1.setText("");
    resetLabel(lblNameError);
    resetLabel(lblSurnameError);
    resetLabel(lblEmailError);
    resetLabel(lblStNumError);
    resetLabel(lblCellNumError);
    resetLabel(lblDobError);
    resetLabel(lblGenderError);
}
```

7. GitHub repository

The full, clickable link to your repository, plus a short note on how a marker can clone and run the project.

Repo Link: <https://github.com/EC-Prime/Input-Validation-System>

To run the project:

Run the pre-built executable (fastest): clone the repository (git clone <https://github.com/EC-Prime/Input-Validation-System>.git) navigate to the dist folder, and double-click ValidationSystem.jar (or run java -jar ValidationSystem.jar from a terminal in that folder). This runs the application as a standalone desktop program without needing to open or build the project in an IDE.

Run from source: clone the repository, open the project folder in Apache NetBeans (the IDE used to build it), and run MainFrame.java directly via the IDE's Run button (or compile with javac / run with java), and run MainFrame's main() method.

8. Conclusion

One of the trickiest parts for me was understanding constructor placement, specifically why Validator needed a private constructor when I wasn't planning to ever create a Validator object in the first place. It took a while to grasp that a constructor isn't the same as a regular method (no return type, same name as the class), and that writing private Validator() {} wasn't adding functionality, it was actively preventing something (anyone calling new Validator()) that Java allows by default. Once that clicked, it changed how I thought about the class as a whole: it wasn't an object with data, it was a toolbox of related functions.

Incorporating OOP efficiently was a bigger struggle than I expected going in. I initially planned around interfaces and a separate model class, but didn't fully understand those concepts well enough to use them confidently, so I made the call to simplify: plain classes and methods only, with Validator handling the reusable rules and MainFrame handling the field-specific logic. Learning to separate "what checks exist" from "which checks apply to which field" was the actual OOP lesson here, even without interfaces, each class has one clear job, which I understand now matters more than which specific Java feature you use to achieve it.

Regex for input formatting was the least intuitive part by far, particularly getting the email pattern right. Reading a pattern like `^[w.+-]+@[w-]+\.[a-zA-Z]{2,}$` character by character and understanding what each symbol was actually matching took real repetition, it wasn't something I could guess at, I had to work through what "something@something.something" looks like as a pattern piece by piece. I came away from it understanding regex well enough to read and adapt a pattern, even if writing one from scratch still isn't something I'd do confidently yet.

9. References

What is RegEx? | Introduction to Regexes | Regular Expressions and Java Patterns | Geekific

<https://youtu.be/HvryGBPmNI8>

Java Classes & Objects

<https://youtu.be/IUqKuGNasdM>

Java access modifiers: (public, protected, private)

https://youtu.be/T632kAJ_9VA

Claude AI for deeper syntax explanation and code structuring/refactoring